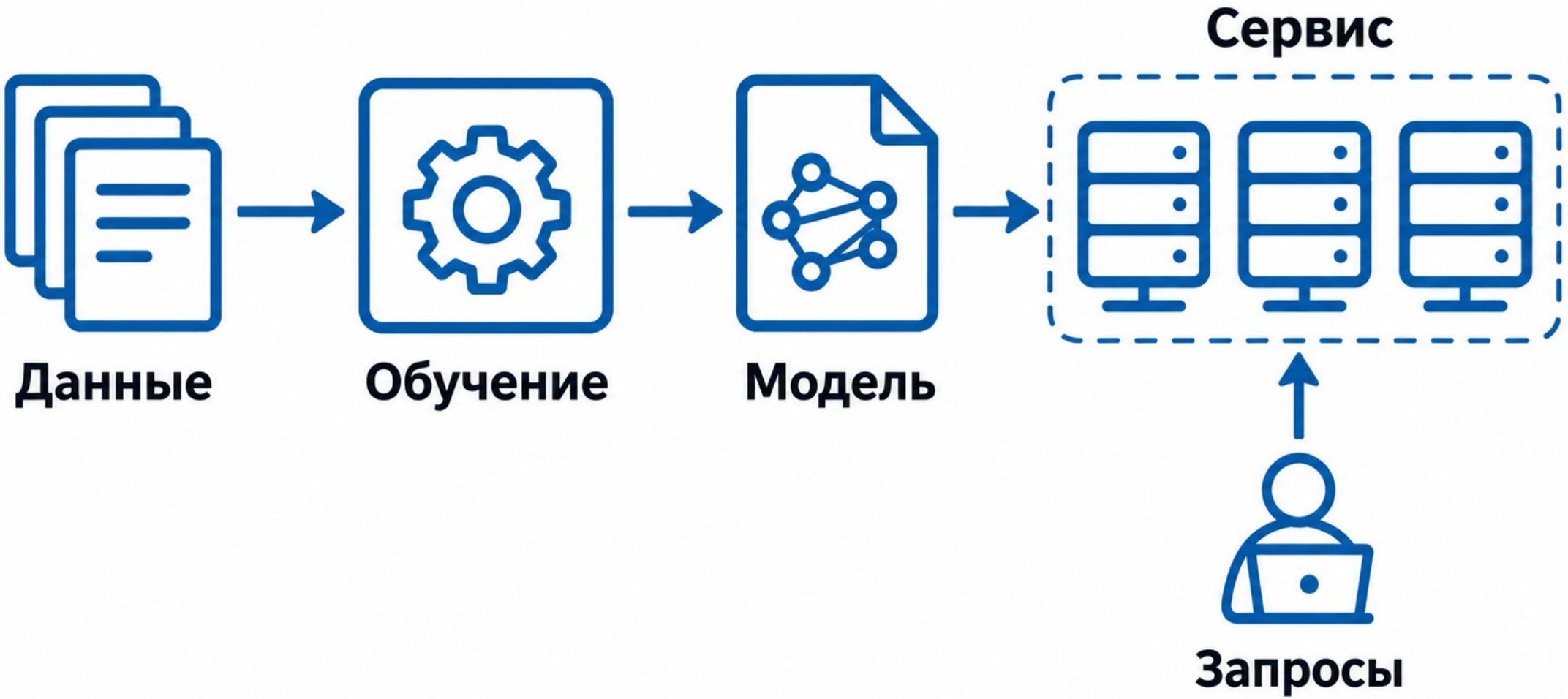
Автоматизация и масштабирование ML-процессов



Сегодня на занятии

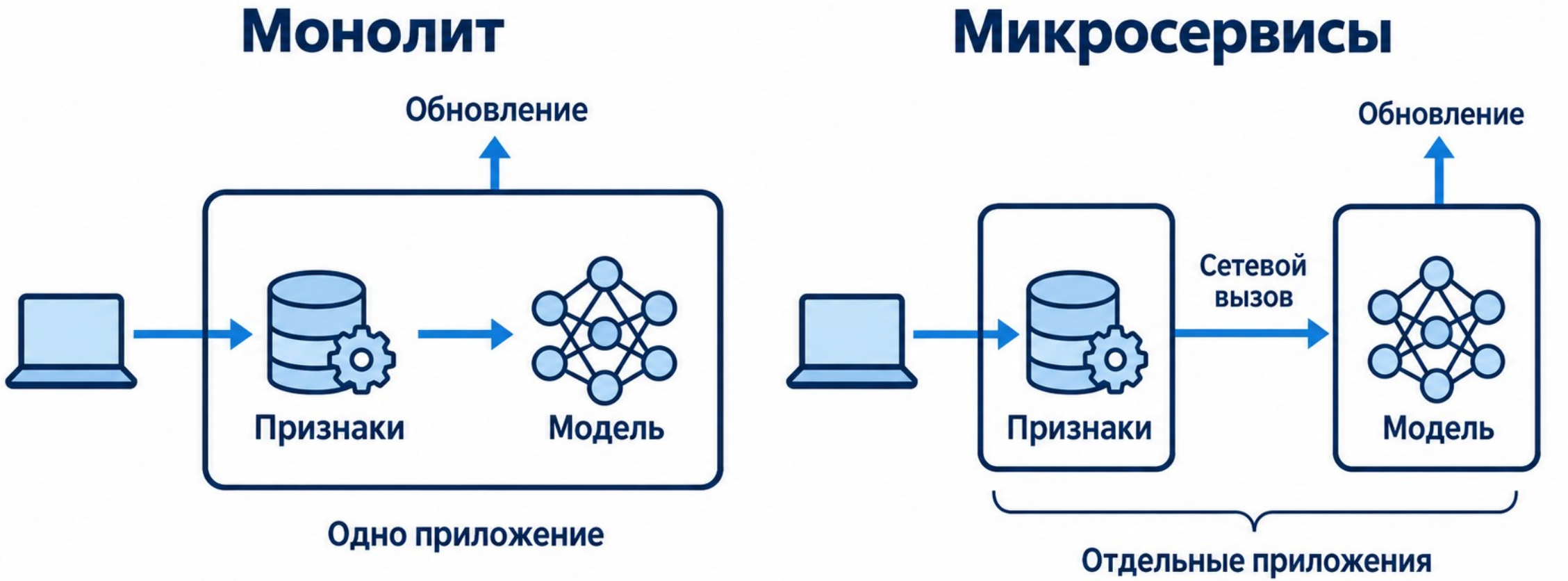
- Окружение и изоляция
- Нагрузка и реплики
- Интерфейс модели
- Задачи и зависимости

Обновление модели в сервисе



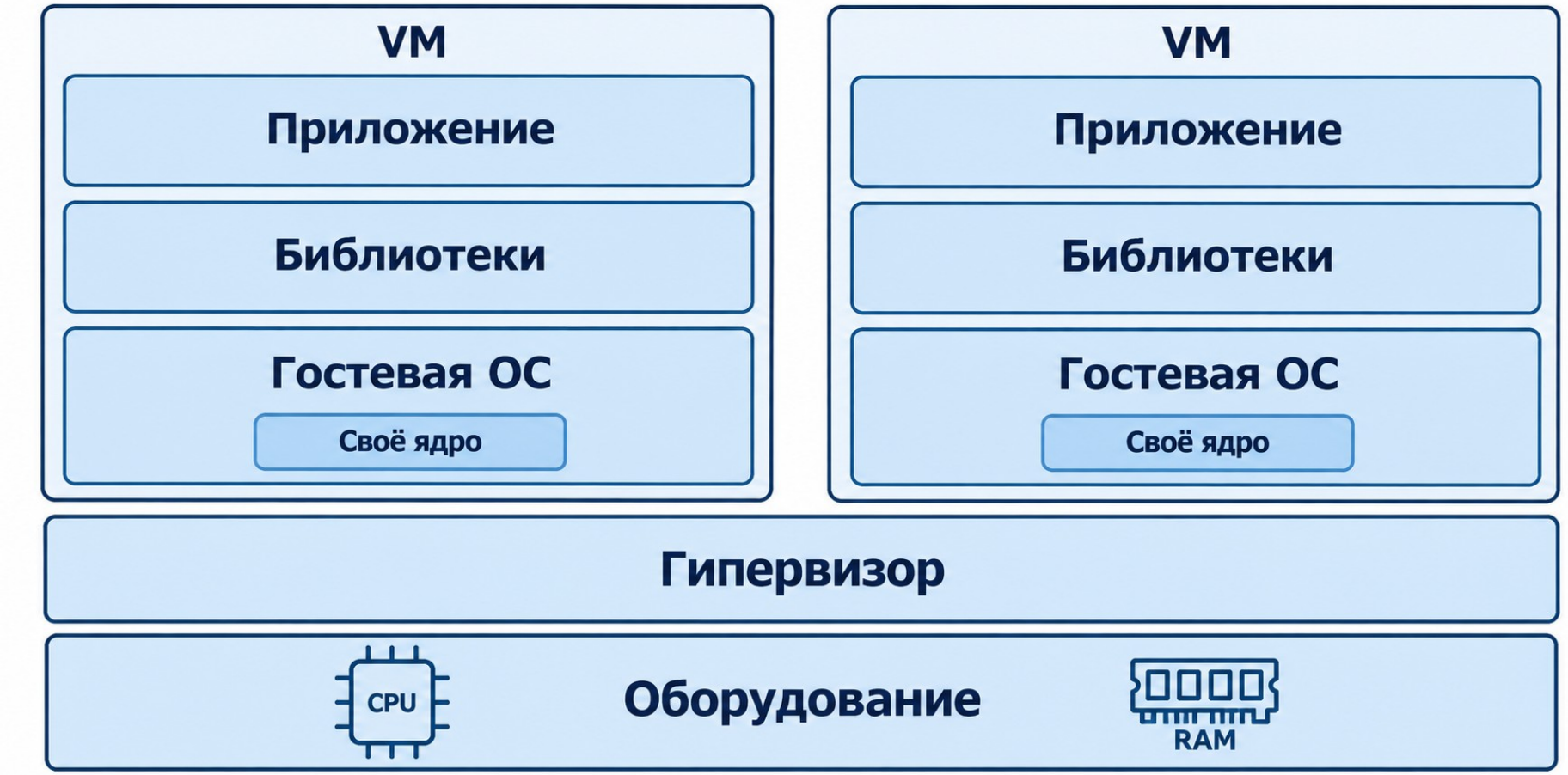
Новая модель → обновление сервиса.

Монолит и микросервисы



Обновляем приложение целиком или отдельный сервис.

Виртуализация: отдельные ОС



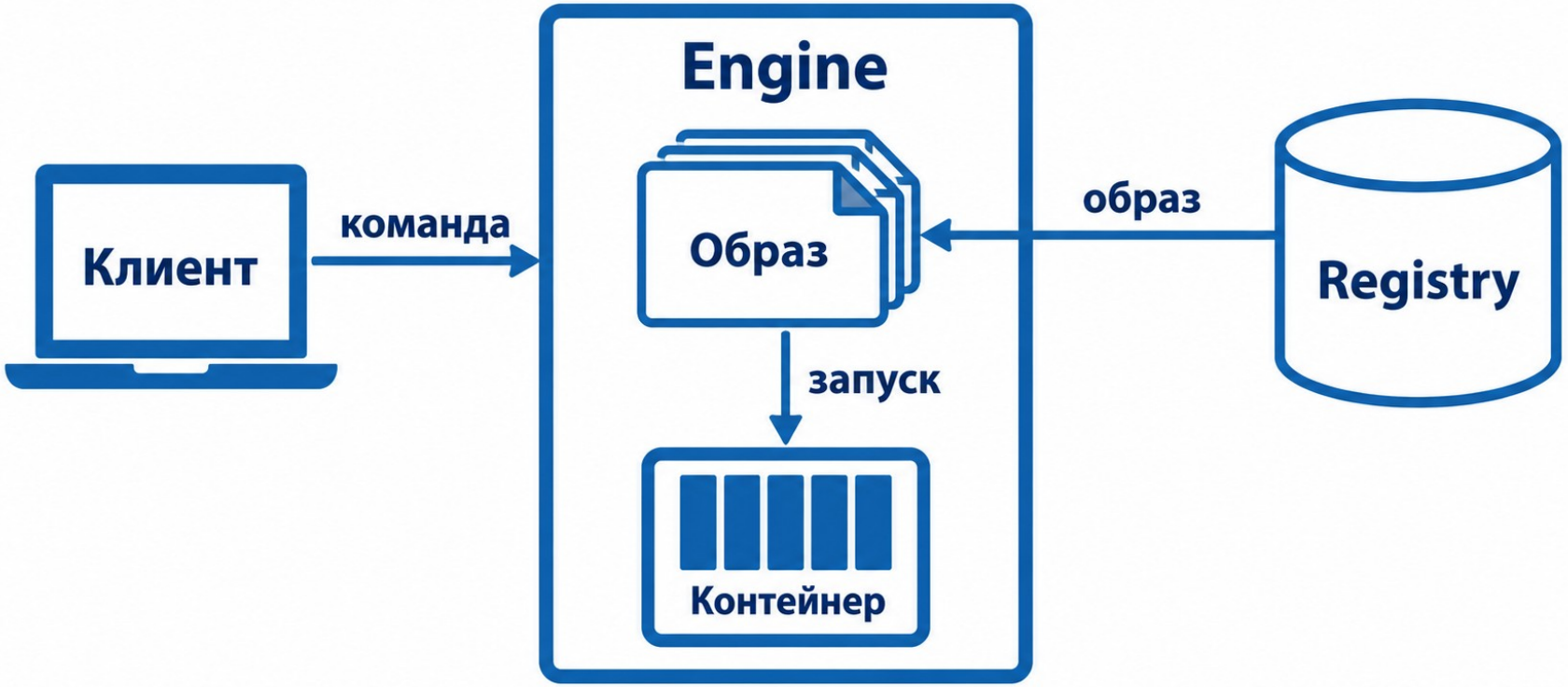
У каждой VM своя ОС.

Контейнеризация: общее ядро



Разные окружения, общее ядро.

Docker: что и зачем

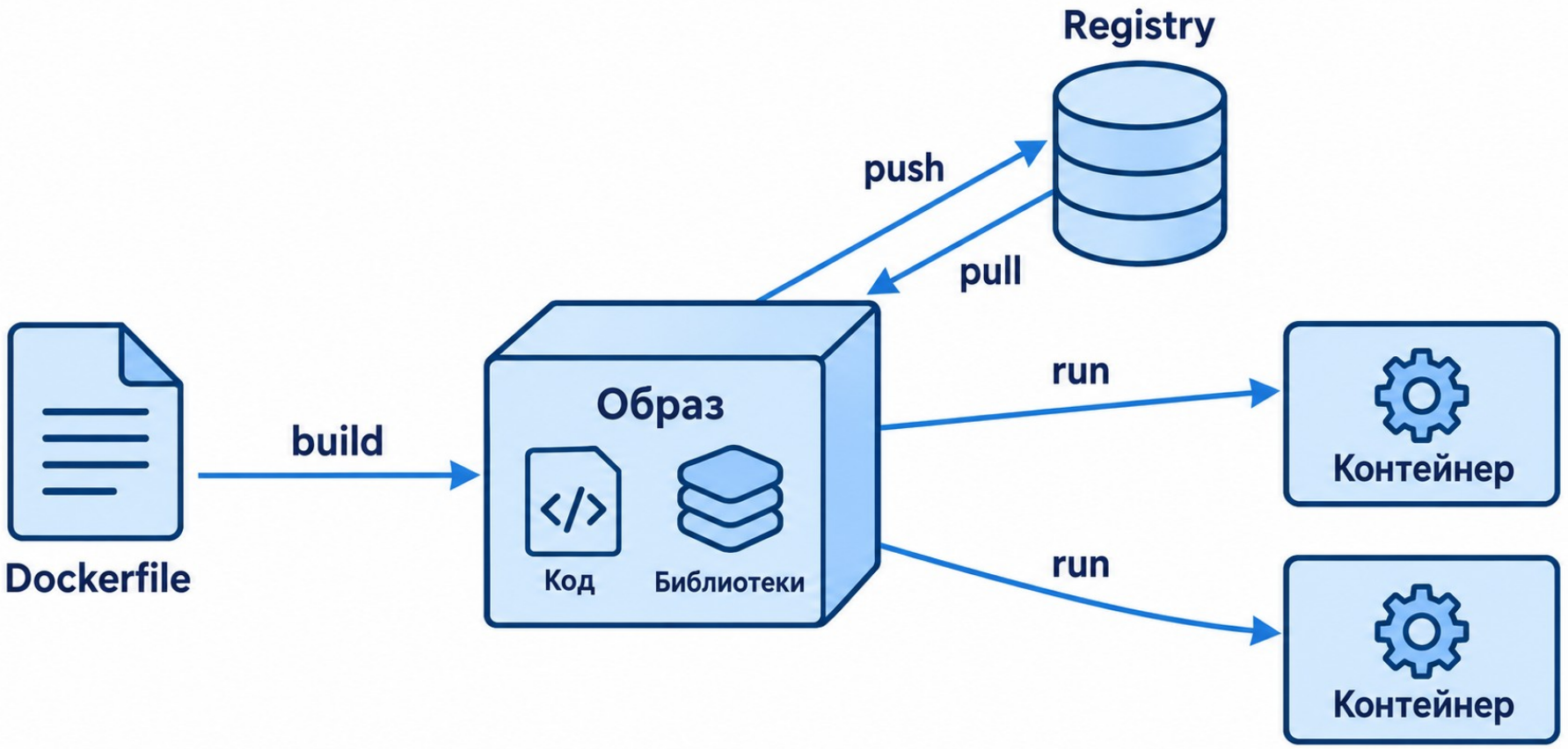


Docker упаковывает окружение и запускает контейнеры.

Клиент → Engine → запуск из образа.

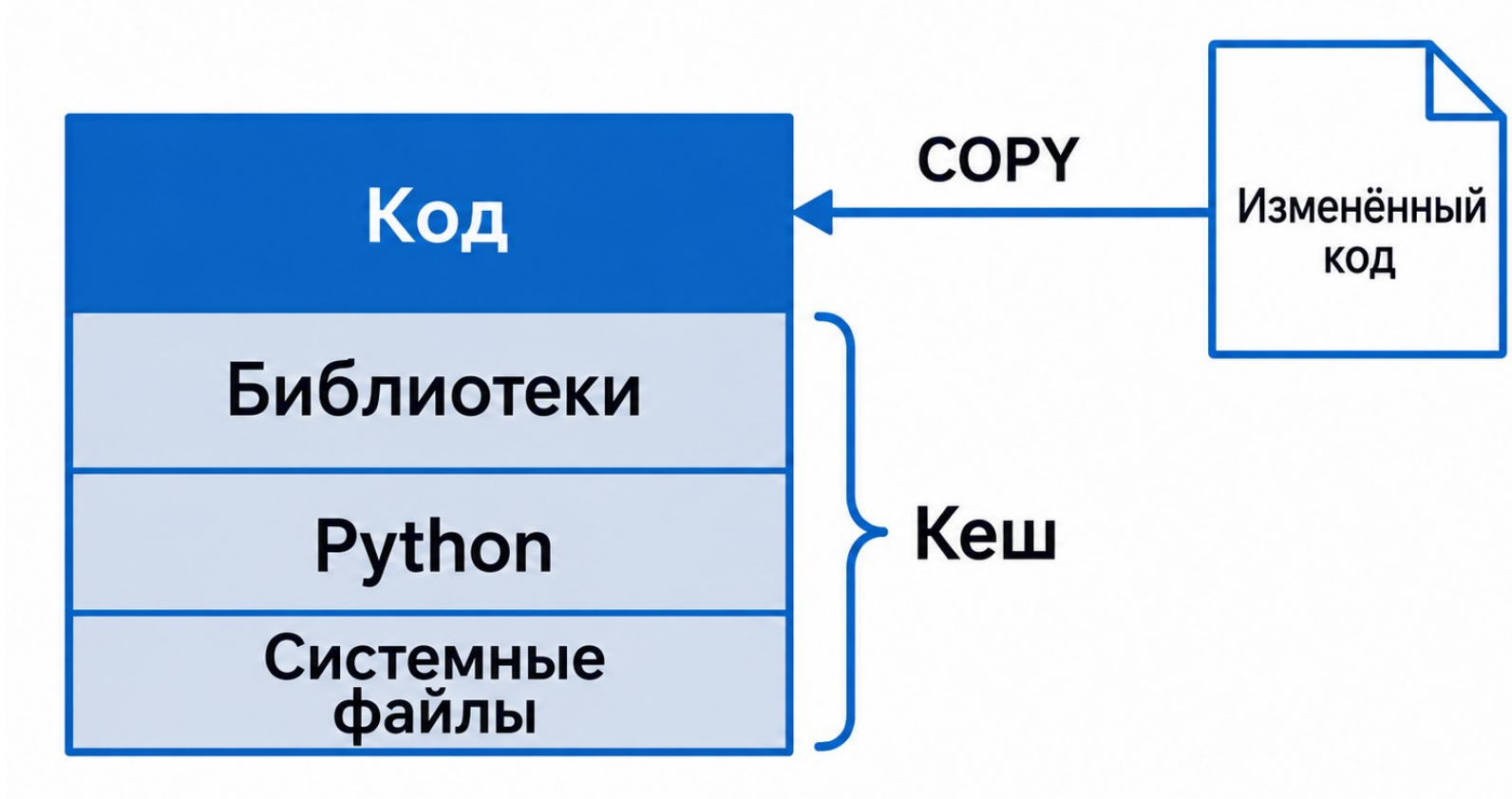
Зачем: переносить одну сборку между машинами.

Сборка, передача и запуск



Собираем образ, передаём через registry, запускаем контейнеры.

Слои образа



Слой: изменения файлов. Кеш: повторное использование.

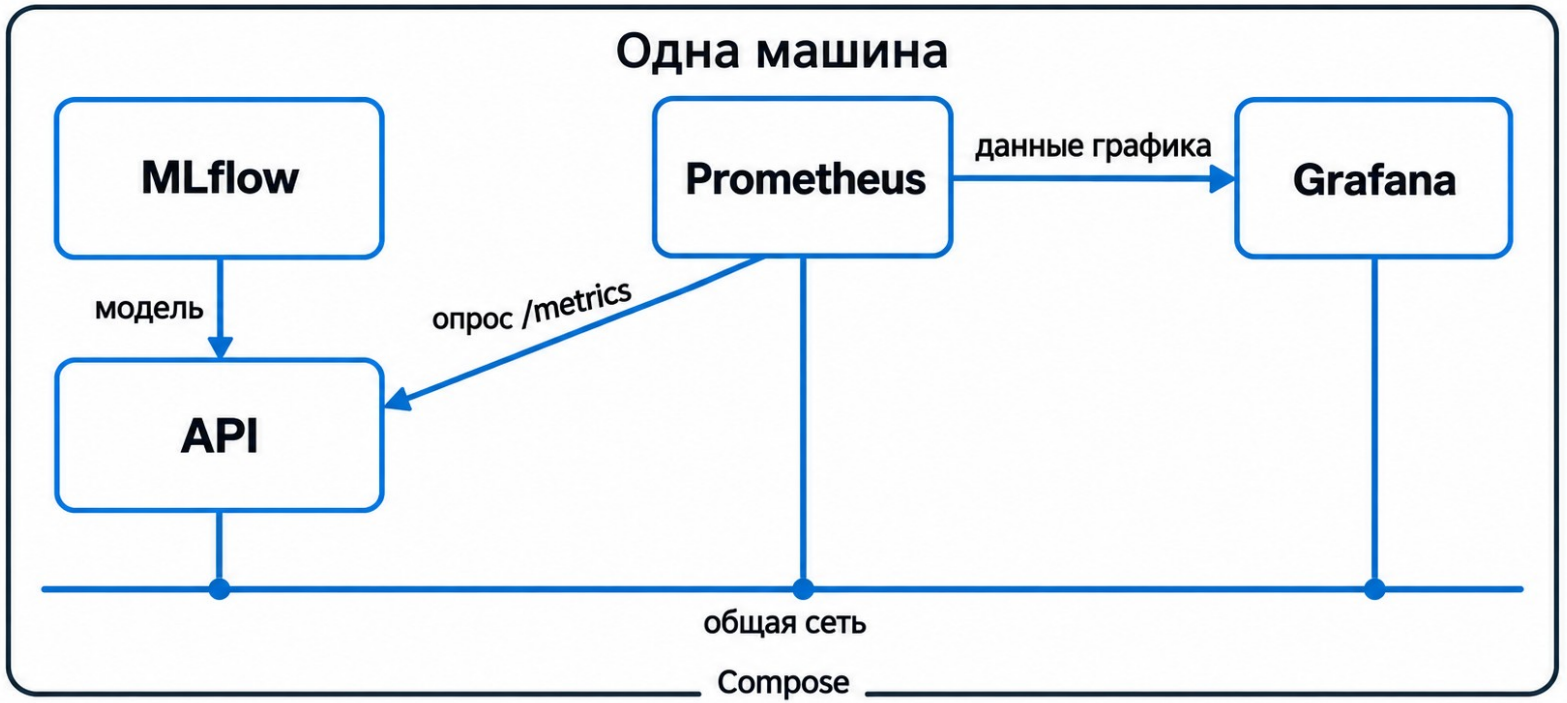
Зачем: не пересобирать неизменившиеся шаги.

Том и папка хоста



volume — том Docker; bind mount — папка хоста.

Compose: общий стек

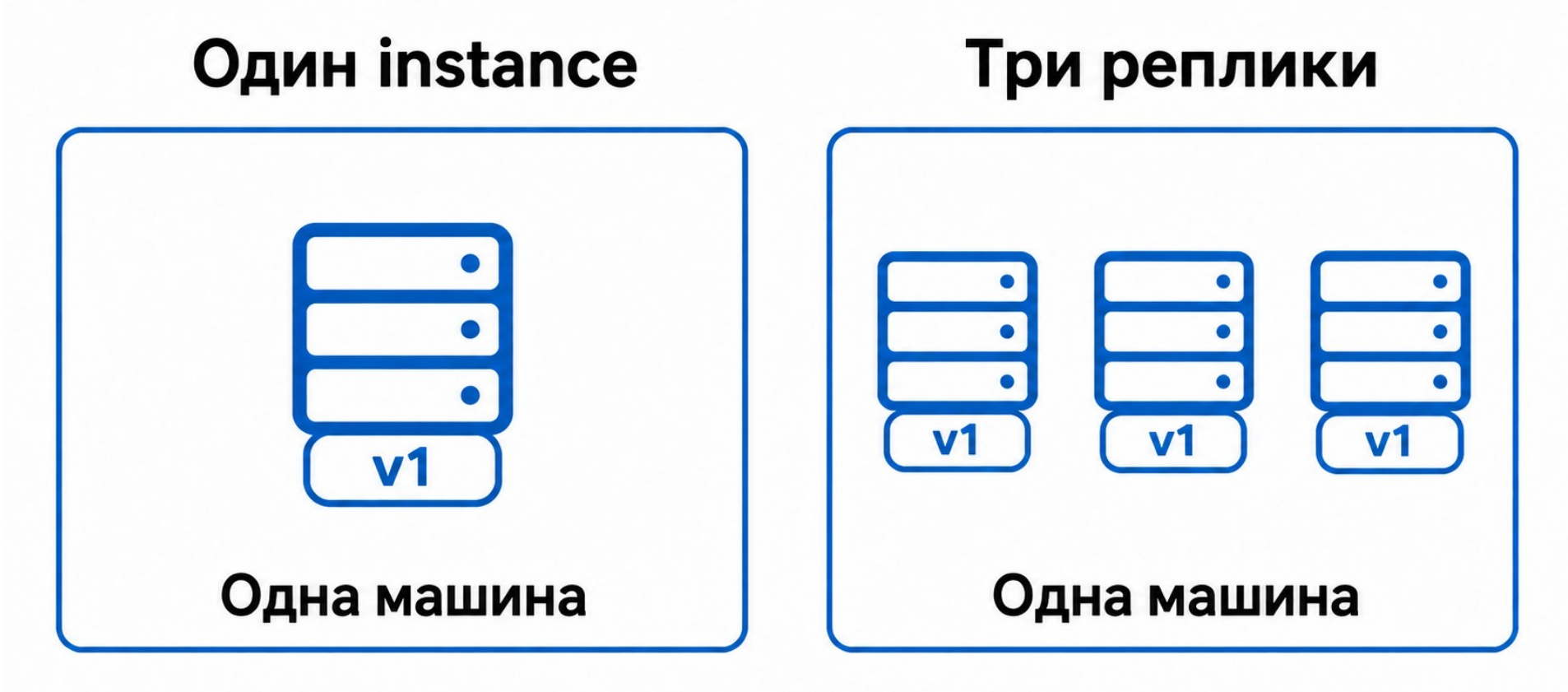


API: интерфейс обращения к приложению.

Compose запускает сервисы и создаёт сеть.

Зачем: связать компоненты.

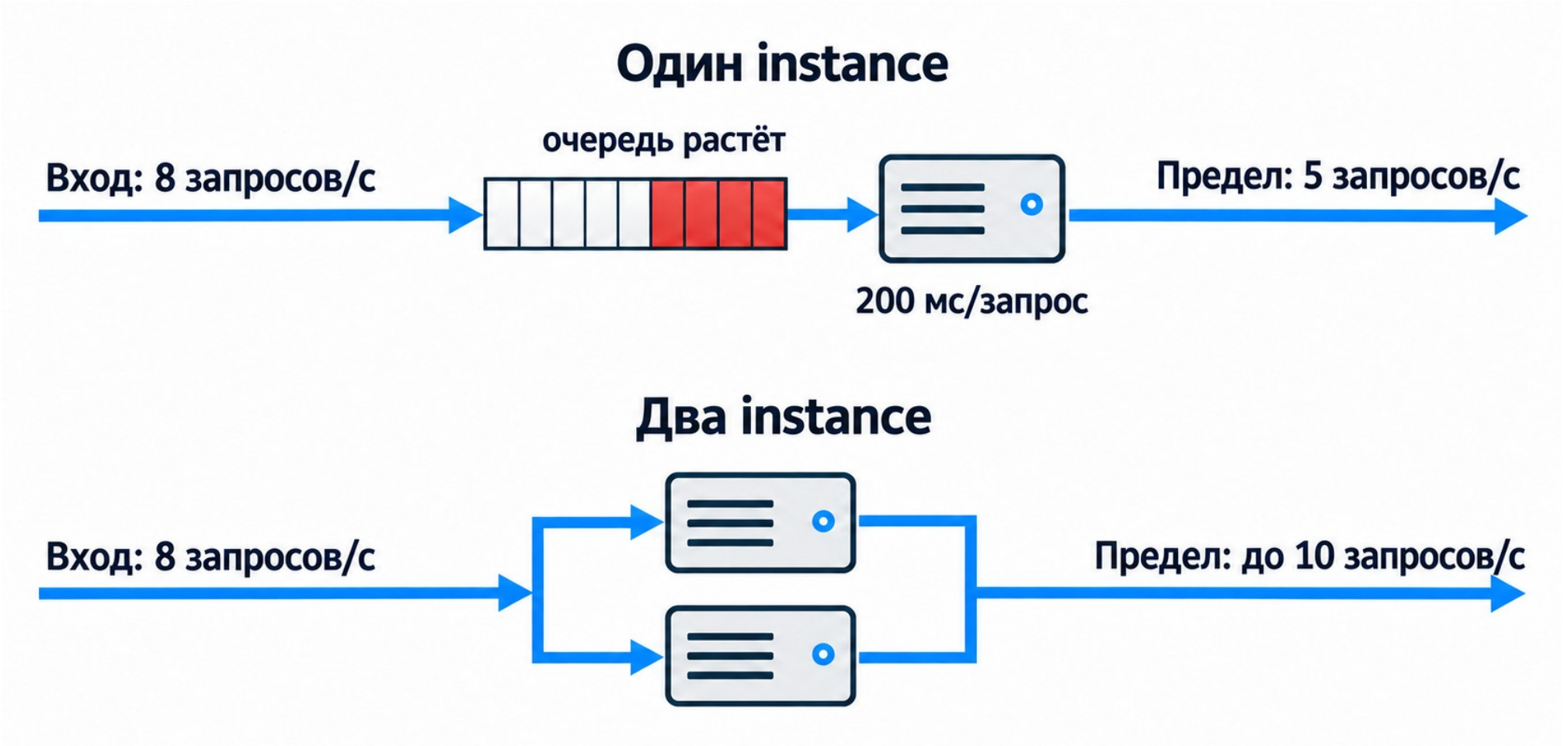
От instance к репликам



Реплика: ещё один instance сервиса.

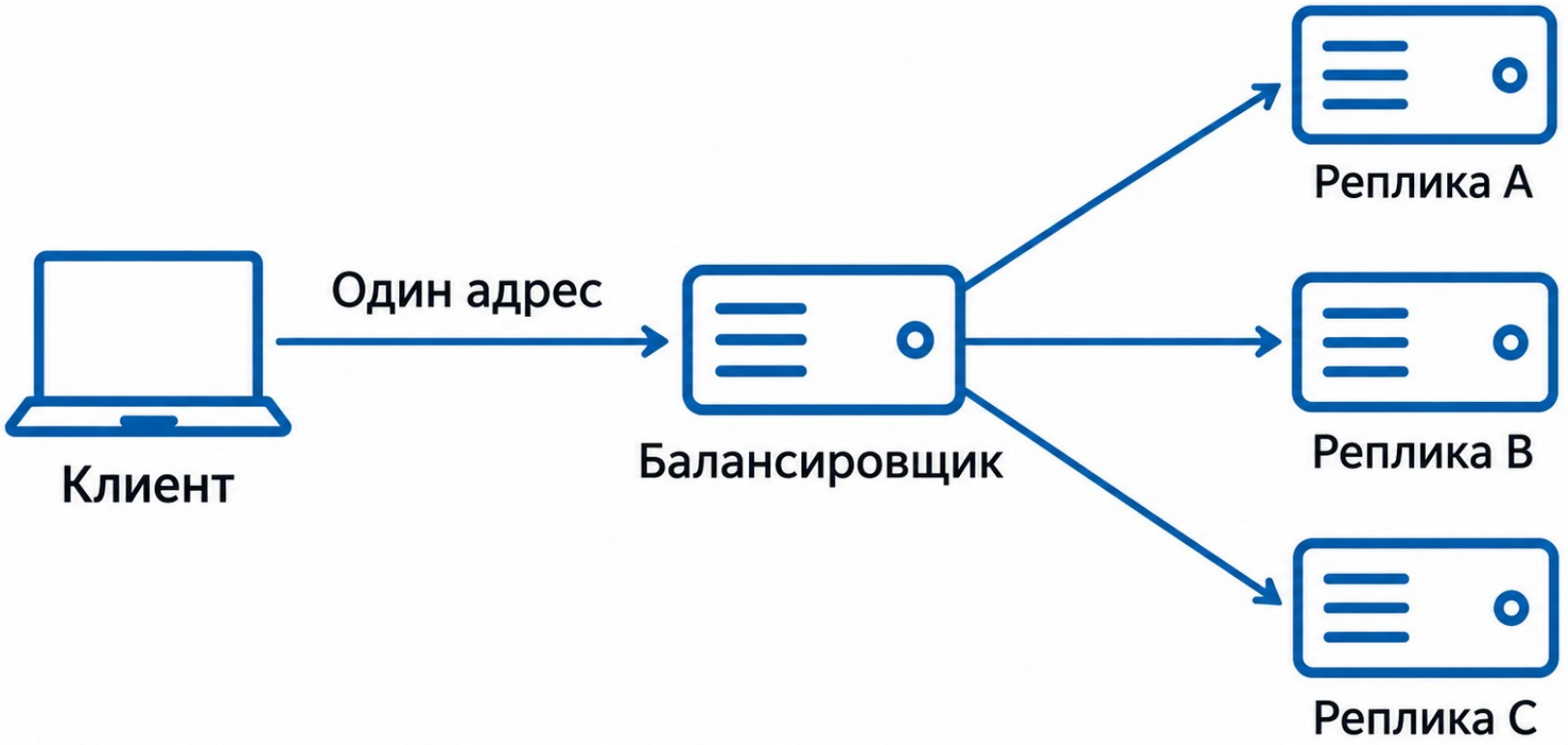
Зачем: распределять нагрузку и переживать отказ instance.

Нагрузка и число instance



При достаточных ресурсах

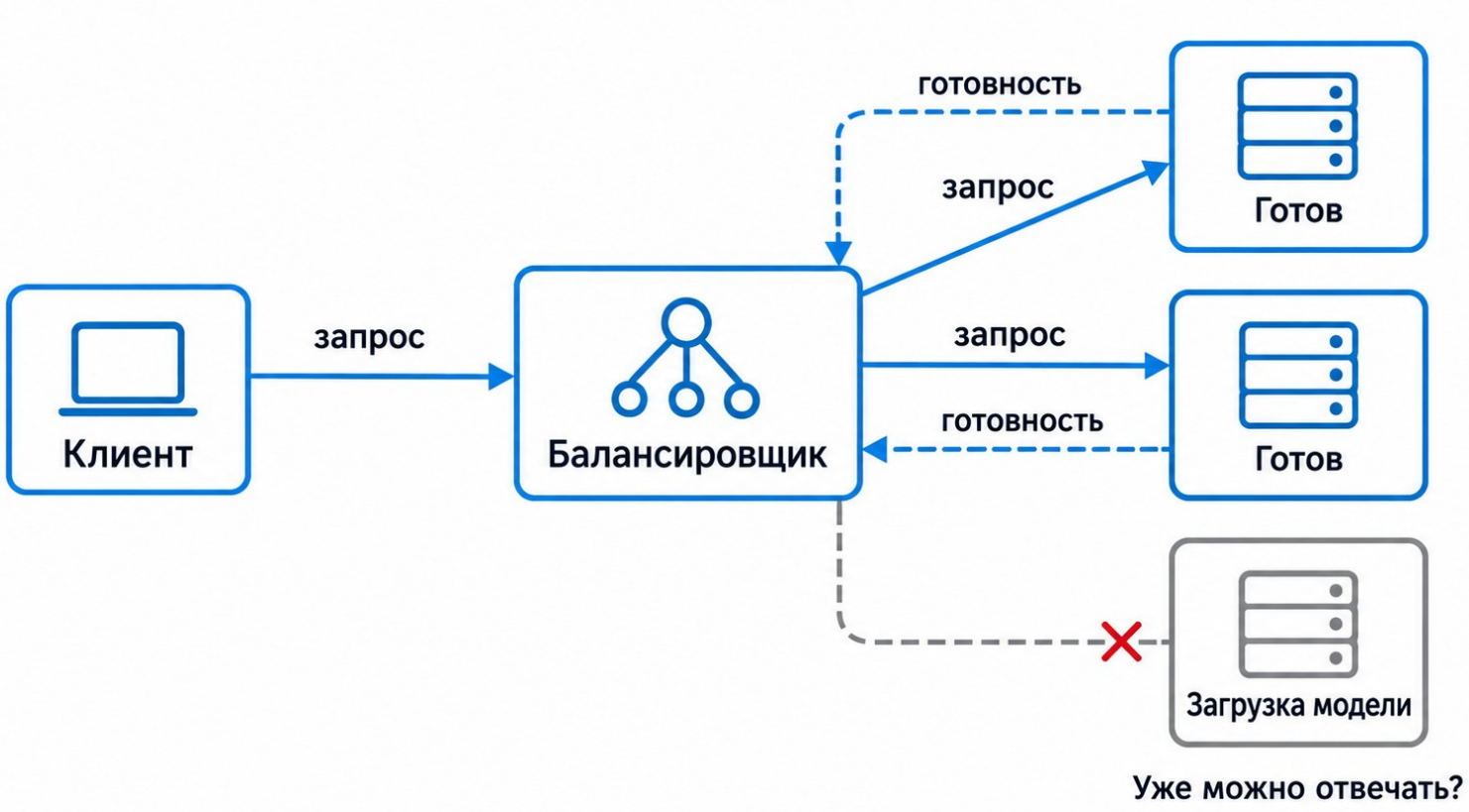
Балансировка запросов



Балансировщик выбирает готовую реплику для запроса.

Зачем: один адрес для нескольких instance.

Запущен ещё не значит готов

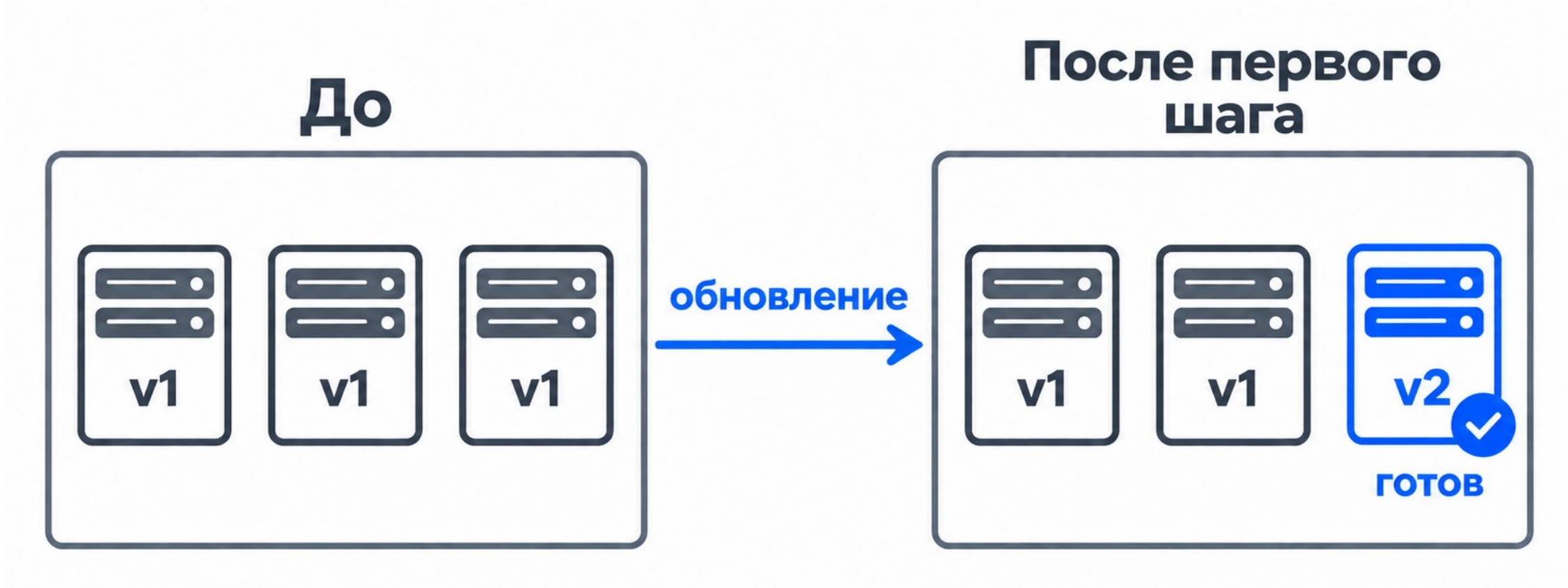


Готовность: сервис отвечает по контракту.

Контракт: ожидаемые входы и выходы.

Зачем: направлять запросы только готовым.

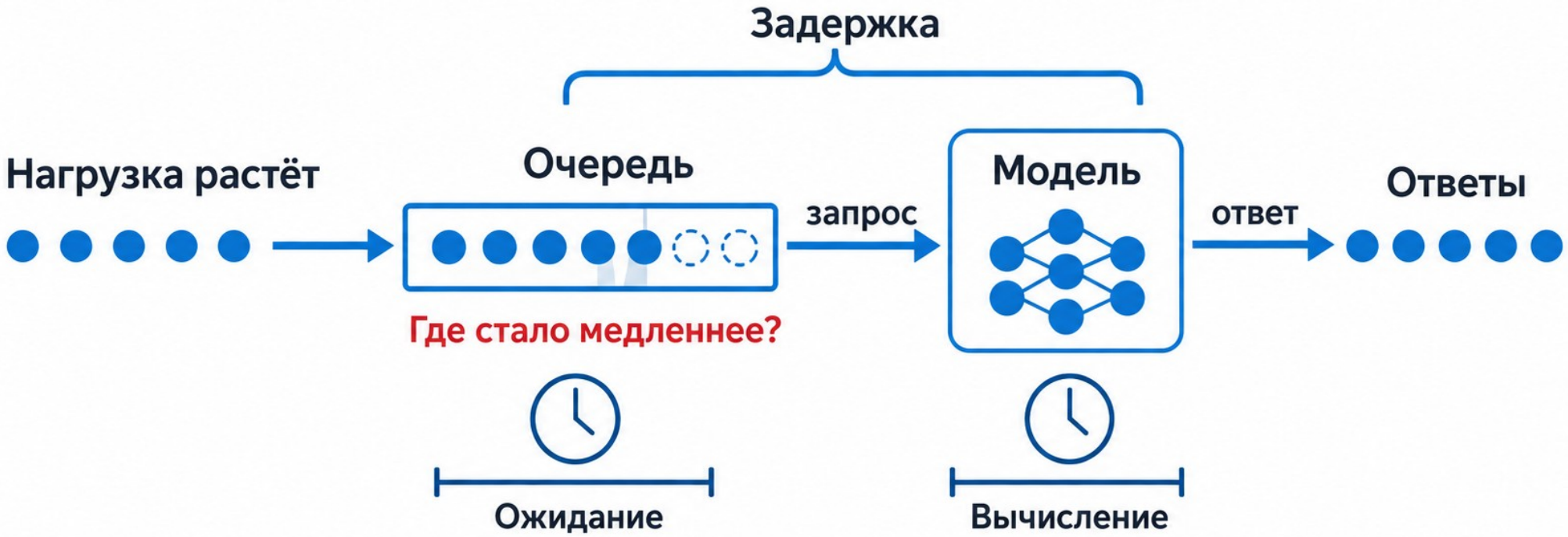
Постепенное обновление



Постепенный выкат: замена реплик по одной.

Откат: возврат предыдущей версии при проблемах.

Из чего складывается задержка



Задержка: время ответа. Пропускная способность: запросы/с.

Очередь: запросы, ожидающие обработки.

При перегрузке растёт ожидание.

gRPC и Protobuf

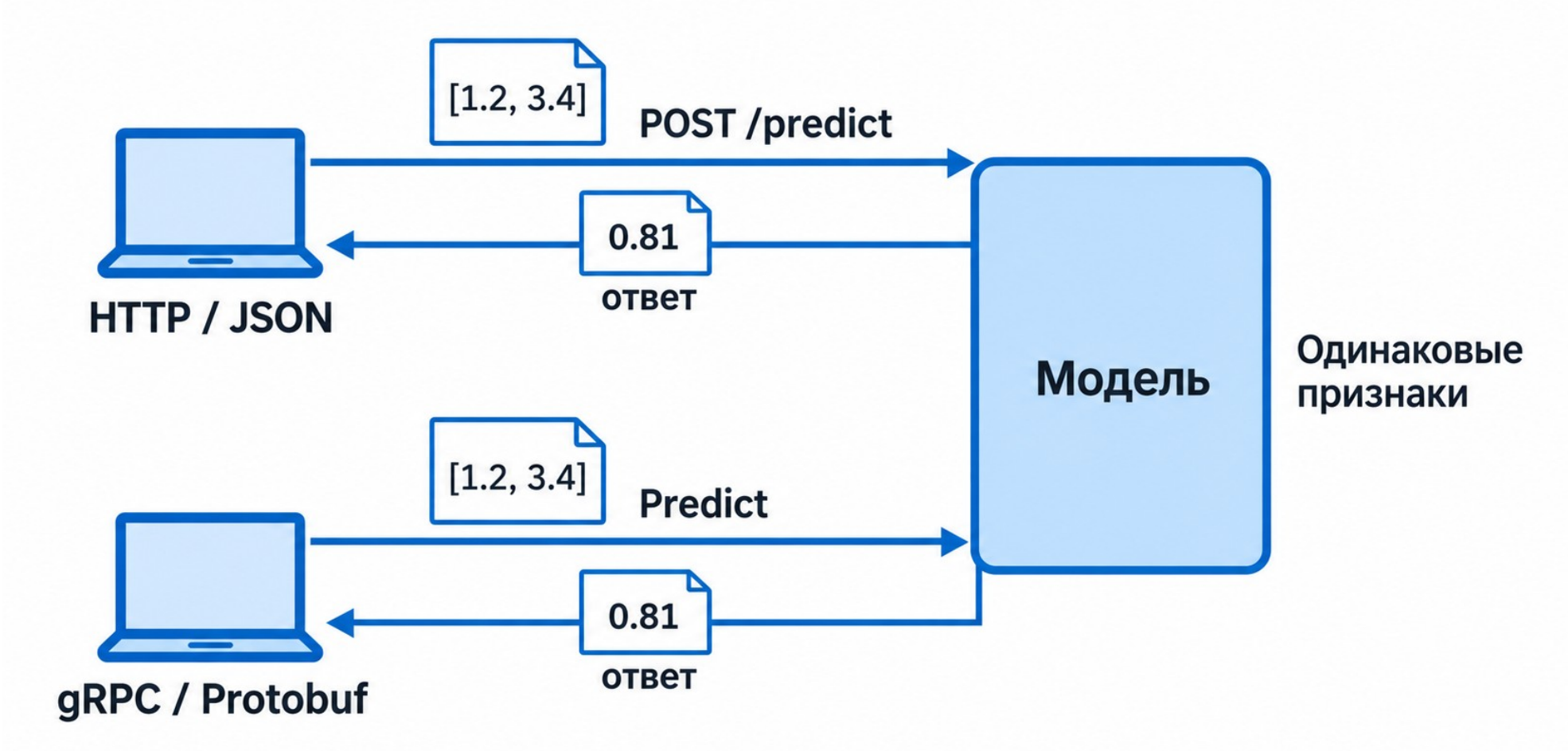


gRPC: удалённый вызов; Protobuf: структура сообщений.

Код клиента генерируется из описания.

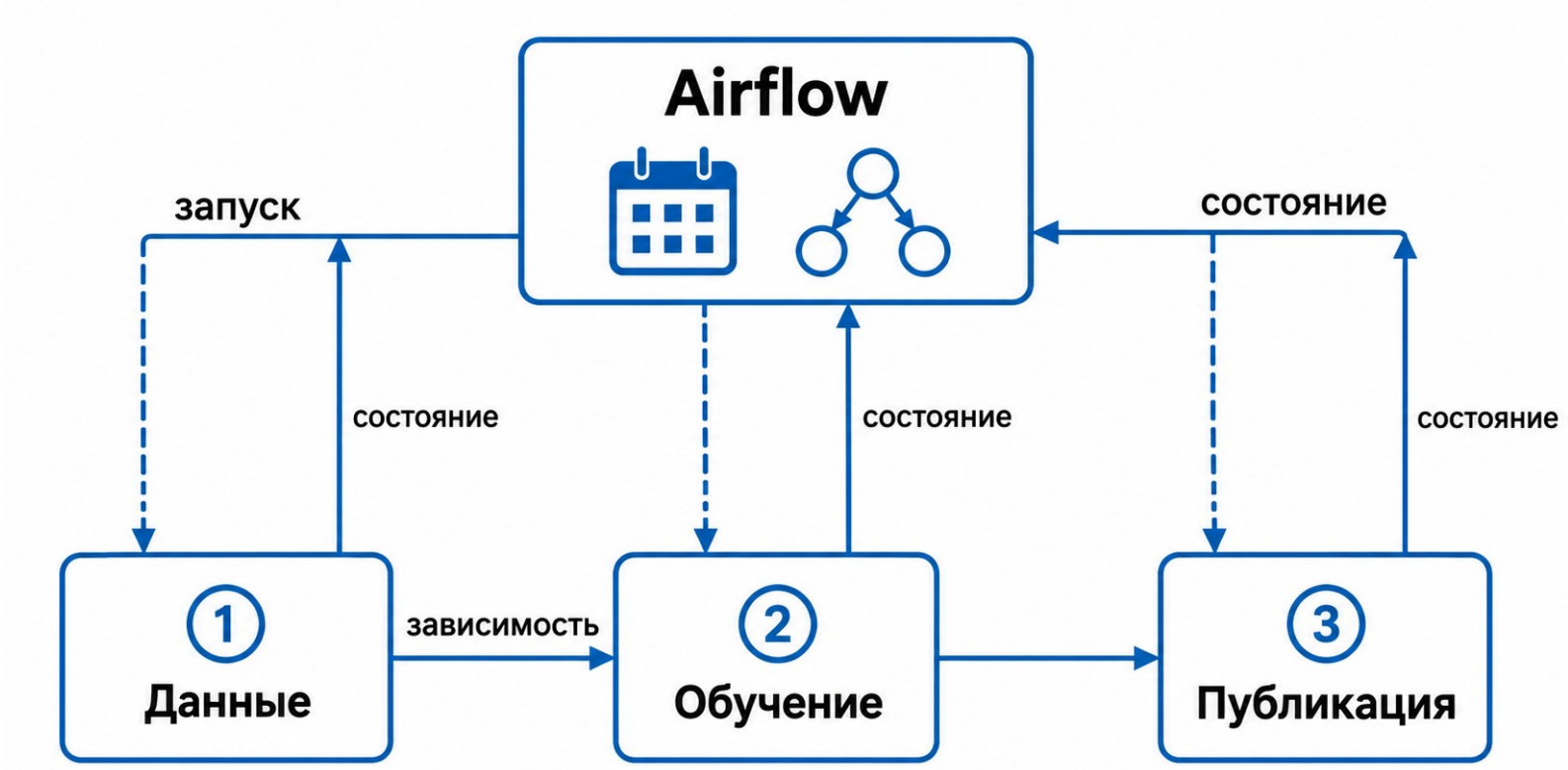
Зачем: согласовать обмен сервисов.

HTTP и gRPC



Контракт: два числа → вероятность класса

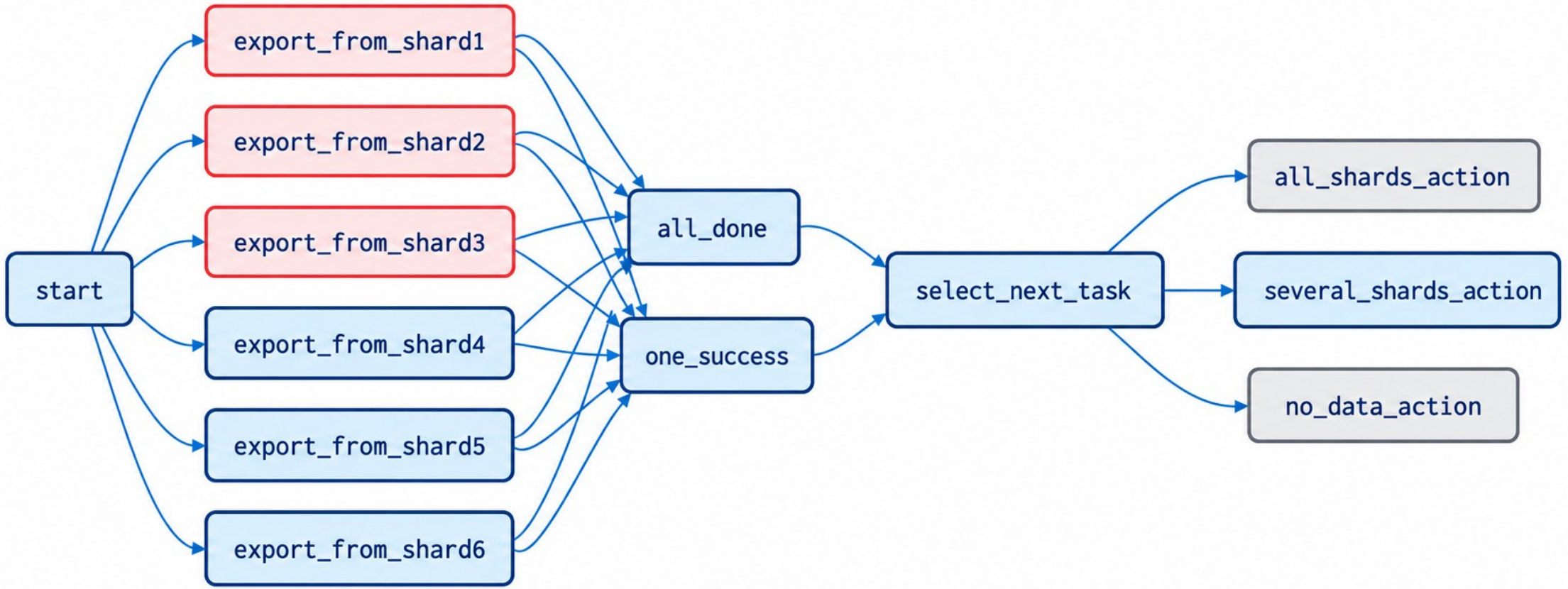
Airflow: управление задачами



Оркестратор запускает задачи по расписанию и зависимостям.

Зачем: автоматизировать порядок и повторы.

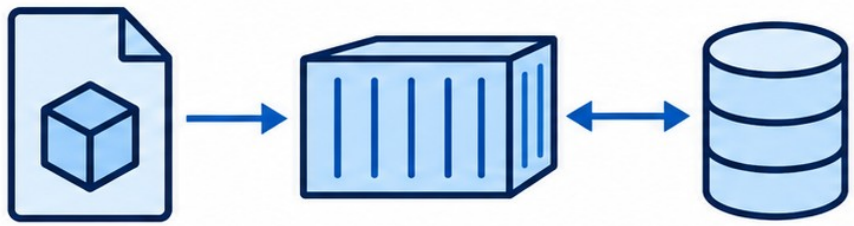
DAG: зависимости задач



- Задачи — шаги работы
- Операторы — действия задач
- Зависимости — порядок запуска
- Расписание — время запуска

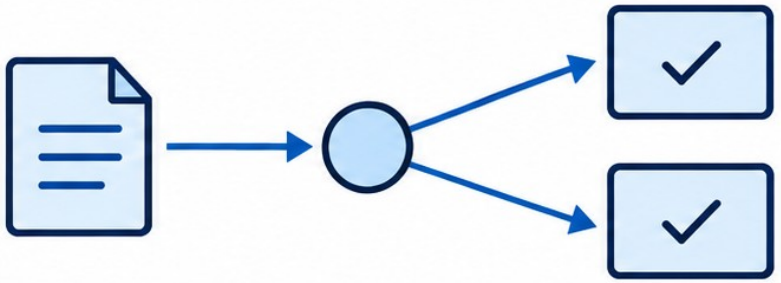
Итоги

Окружение



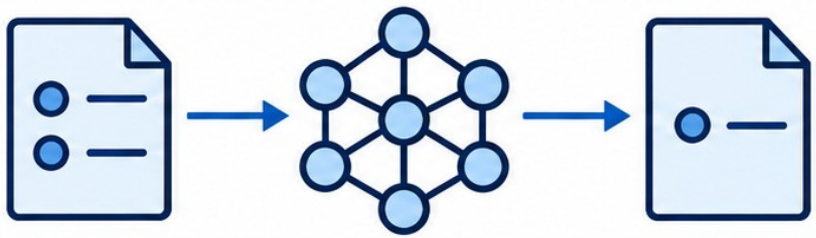
Образ фиксирует окружение
Том сохраняет данные

Нагрузка



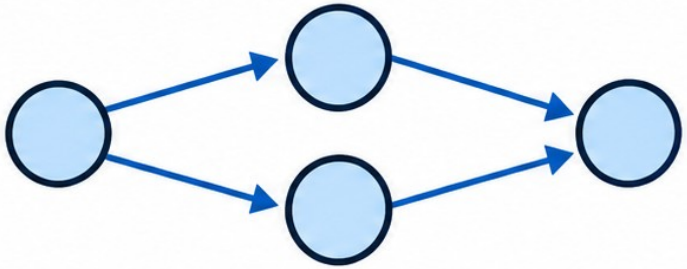
Запросы получают готовые instance

Интерфейс



Входы и ответы согласованы

Задачи



DAG задаёт зависимости